

5

Hardening the Context Engine

In [Chapter 4](#), we built a working prototype of the Context Engine, a system powered by a team of specialist agents and an intelligent orchestrator guided by a LLM-driven Planner. The resulting notebook, `Context_Engine.ipynb`, validated our architectural blueprint and proved that concepts such as dynamic planning and context chaining work not just in theory. However, a prototype is not a production-ready system. The current engine is innovative but lacks the modularity and transparency required for real-world deployment. This chapter focuses on transforming our experimental prototype into a business-ready pre-production framework. This transformation unfolds across three stages.

We begin with an architectural dive, examining the sequential structure of the upgraded engine. Next, we move into component hardening, strengthening every part of the system with production-grade practices such as structured logging, dependency injection, and error handling. Finally, we conclude with refactoring for modularity, converting the single, monolithic notebook into a clean, scalable application built from reusable library components. By the end of this chapter, the Context Engine will no longer be an experiment. It will be a hardened, transparent scalable system ready to support complex, multi-agent workflows. This chapter will cover the following main topics:

- Step-by-step architectural walk-through
- Refactoring for scalability
- Modularizing the engine

Step-by-step architectural walk-through

Before we assemble the code for the upgraded Context Engine we will build in `Context_Engine_Pre_Production.ipynb`, it's critical to understand its internal mechanics. Our guide for this exploration will be *Figure 5.1*.

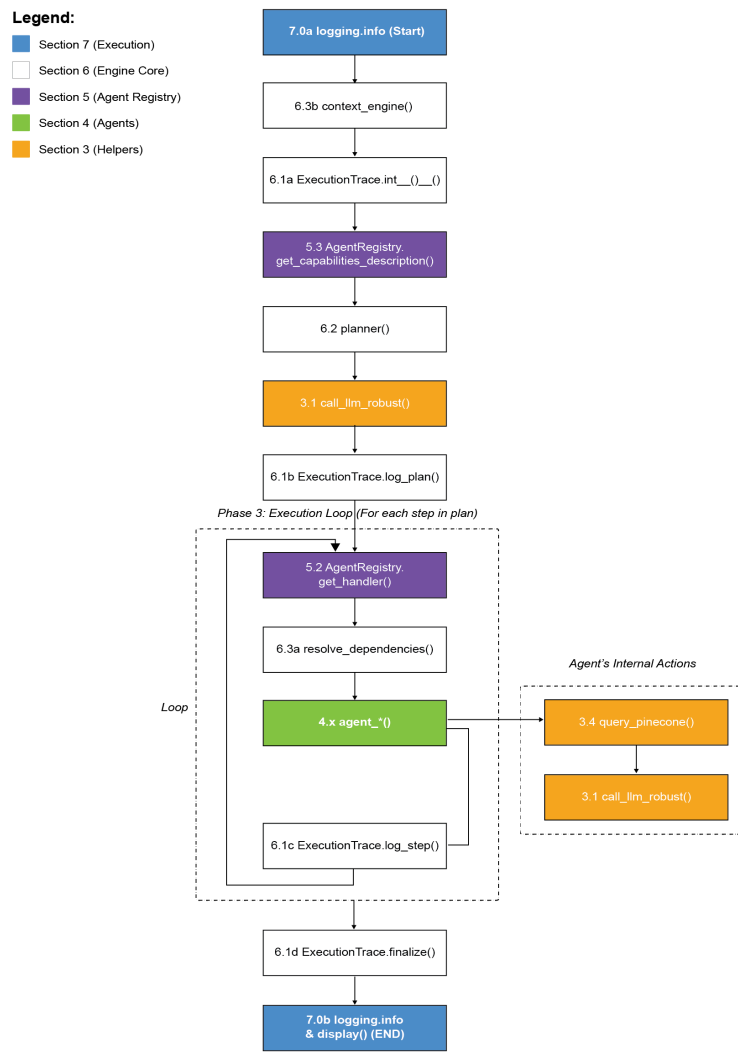


Figure 5.1: The upgraded Context Engine flowchart

Figure 5.1 traces a single task end to end as it moves through the system. Each node is color-coded to match the code sections in `Context_Engine_MAS_MCP.ipynb`, where that function lives, so you can see the logic thread across the code base. The notebook also contains the upgrades we will be making in the *Refactoring for scalability* section and tweaking during the validation process:

- **Section 7 (Blue):** The final execution script the user runs
- **Section 6 (White):** The engine core, including the orchestrator, planner, and tracer
- **Section 5 (Purple):** The Agent Registry, the system's toolkit
- **Section 4 (Green):** The specialist agents that do the actual work
- **Section 3 (Orange):** Shared helpers for LLM calls, embeddings, and vector database queries

The engine's operation unfolds across four distinct and interdependent phases: **Initiation**, **Planning**, **Execution**, and **Finalization**. Each phase performs a precise role in the overall workflow, passing control to the next. Together, these stages transform a user's goal into a context-aware output.

Please note that throughout this walk-through, the section numbers and colors in parentheses mirror the labels in Figure 5.1.

Phase 1: Initiation

Every operation begins with a user action, the moment a goal is submitted and the engine is set in motion.

- **7.0a** `logging.info` (Blue): The very first action is to log the start of the process. This is a simple but crucial step in any production-oriented system for tracking tasks.
- **6.3b** `context_engine()` (White): This is the main entry point and the heart of the orchestrator. The user's goal is passed into this function, which now takes control of the workflow from this point forward.

Phase 2: The planning stage

Once inside `context_engine()`, the first task is to create a strategy. The engine doesn't act blindly; it thinks first. The engine gathers information about what tools it has, what the user wants, and how to bridge the two. It then drafts a detailed execution plan that the downstream components can follow.

- **6.1a** `ExecutionTrace.__init__()` (White): The engine immediately initializes an `ExecutionTrace` object. This object acts as the engine's "flight recorder," ready to log every subsequent action for transparency and debugging.
- **5.3** `AgentRegistry.get_capabilities_description()` (Purple): To create a valid plan, the Planner needs to know what tools it has available. It queries the `AGENT_TOOLKIT` object to get a plain-text description of all specialist agents and their required inputs.
- **6.2** `planner()` (White): With the user's goal and the list of agent capabilities in hand, the engine calls the `planner()` function.
- **3.1** `call_llm_robust()` (Orange): The `planner()` function delegates the reasoning task to an external LLM. It sends the goal and capabilities in a carefully structured prompt and asks the LLM to return a strategic, multi-step JSON plan.
- **6.1b** `ExecutionTrace.log_plan()` (White): As soon as the JSON plan is received from the LLM, it's logged to our `ExecutionTrace` object. This concludes the planning phase.

Phase 3: The execution loop

This is where the Context Engine comes to life. With a plan in hand, the system begins executing each step in strict sequence (where *step* refers to the actual, numbered steps of the dynamic flow of the Context Engine). It retrieves the correct agent for the job, prepares its inputs, and tracks every output, forming a continuous feedback loop of planning, acting, and verifying.

The loop begins with the Executor reading the next step from the plan:

- **5.2** `AgentRegistry.get_handler()` (Purple): The plan specifies an agent by name (e.g., "Librarian"). The Executor uses `get_handler()` to retrieve the actual, callable Python function (e.g., `agent_context_librarian`) from the registry.
- **6.3a** `resolve_dependencies()` (White): This is the core of context chaining. The Executor checks the inputs required for the agent. If it finds a placeholder like `$$STEP_1_OUTPUT$$`, this function replaces it with the actual data produced by *Step 1* (the first step of the dynamic flow of the Context Engine), which is stored in the engine's `state` dictionary.
- **4.x** `agent_*` (Green): The Executor now calls the retrieved agent function, passing it the fully resolved context.

Within each agent, specialized logic drives the system's problem-solving behavior. These agents often rely on helper functions to access external resources and perform the heavy lifting:

- The Librarian and Researcher agents call **3.4** `query_pinecone()` (Orange) to perform semantic searches on the vector database. This function, in turn, uses **3.2** `get_embedding()` (Orange) to convert the text query into a vector.
- The Researcher and Writer agents call **3.1** `call_llm_robust()` (Orange) to either synthesize facts or generate the final content.
- **6.1c** `ExecutionTrace.log_step()` (White): Once the agent completes its work and returns its output, the `log_step()` method is called to record everything about the completed step: the agent used, its inputs, and its final output. The output is also saved to the engine's `state` to be used by subsequent steps.

The loop then proceeds to the next step in the plan, repeating the entire sequence of retrieving the agent, preparing its inputs, executing the step, and logging the outcome.

Phase 4: Finalization

The final phase closes the loop. After all steps have executed successfully, the Context Engine wraps up its task by logging its final status and returning both the result and full trace. This stage is about graceful completion, handing back a clear outcome and a transparent record of how it was achieved.

- **6.1d** `ExecutionTrace.finalize()` (White): The engine calls `finalize()` on the trace object, recording the final status ("Success") and the total execution time
- **7** The `context_engine()` then returns the final output (the result from the very last step) and the complete trace object back to the user's script
- **7.0b** `logging.info & 7.1/7.2 display(Markdown(...))` (Blue): The (Blue): The script logs that the task is complete and displays the final, formatted output to the user, successfully fulfilling the original goal

Each color block in the preceding flowchart represents a well-defined component with a single responsibility. Together, they form a clean, layered architecture, an essential trait of scalable multi-agent systems:

- **The engine core (White)** is the brain. It doesn't do the work itself but manages the entire process. It plans the strategy (planner), executes the plan step by step, and records everything (`ExecutionTrace`). Its main function, `context_engine()`, is the master orchestrator.
- **The Agent Registry (Purple)** is the toolkit. It serves two masters: it provides a descriptive manual (`get_capabilities_description`) to the Planner so it can think, and it provides the actual tools (`get_handler`) to the Executor so it can act.
- **The specialist agents (Green)** are the workers. Each has one specific job: the Librarian finds stylistic instructions, the Researcher finds facts, and the Writer combines them to create content. They are the hands of the operation.
- **The helper functions (Orange)** are the utilities. These are shared, low-level functions that any other component can use. They handle repetitive but essential tasks like communicating with the LLM (`call_llm_robust`) and the vector database (`query_pinecone`), preventing code duplication and centralizing key interactions.

By analyzing this architecture, we can see how the system achieves complex behavior through a logical, predictable, and traceable sequence of simpler operations. Now that we have explored the blueprint in detail, we are ready to begin constructing each of these components in code.

Refactoring for scalability in the real world

Our journey of transforming the prototype into a production-ready application begins with a crucial step in software engineering: refactoring. In [Chapter 4](#), our notebook contained several cells dedicated to installing packages, initializing API clients, and also the helper, agents, and context engine code. While functional, this approach has two significant drawbacks: it clutters our main workspace with low-level setup tasks, and it forces us to repeat the same code in every new notebook we create.

To solve this, we will now move this foundational logic into our new `utils.py` file. Think of this as moving from a cluttered workbench to a professional, organized workshop. Instead of having our tools and keys scattered everywhere, we will create a centralized "setup checklist" that can be run with a single command. This not only cleans up our primary notebook, allowing us to focus on the high-level orchestration of our engine, but it also ensures that every component of our project is built on a consistent and reliable foundation. We will also go through the real-world issues that arise during the validation runs and how we solve them.

[Chapter 4](#) validated the logic of the Context Engine; this section transforms that working concept into a maintainable system. We move from a single, notebook-centric prototype to a modular, dependency-injected, and fully instrumented code base that teams can reproduce.

Open `Context_Engine_Pre_Production.ipynb` and let's begin transforming the context engine into a pre-production system.

Creating the centralized setup functions

The first step is to create two dedicated functions in our `utils.py` file: one to handle installations and another for client initialization.

- The `install_dependencies()` function acts as a "one-click installer" for our project. By encapsulating all the `pip` commands into this single function, we guarantee that every notebook in our project will use the exact same versions of the required libraries. This is critical for ensuring reproducibility and avoiding the dependency conflicts that can often plague complex projects.
- The `initialize_clients()` function serves as our secure keymaster. It centralizes the entire process of securely fetching `API_KEY` and `PINECONE_API_KEY` from Google Colab Secrets and then uses them to create and configure the OpenAI and Pinecone client objects.

Together, these functions give us a reusable and secure setup process, so every component in the project starts from a consistent and reliable environment.

Simplifying the main notebook

With our setup logic now neatly organized in `utils.py`, we can dramatically simplify our main `Context_Engine_Pre_Production.ipynb` notebook. The multiple, verbose cells for installation and client setup are now replaced with a single, elegant cell that gets our environment ready with just a few lines of code.

We first import our new utility module and then simply call the two setup functions we created earlier. This act of abstraction is the essence of professional software design. Our main notebook no longer needs to know the *details* of the setup; it just needs to know that it *must be done*. This

leaves us with a clean, readable workspace, ready to focus on the more advanced tasks of upgrading our engine's core logic.

With the foundation in place, we can now move to the next step: hardening the helper functions, which form the communication backbone of the Context Engine.

Hardening the helper functions

With our project structure refactored, we now turn our attention to the engine's central nervous system: the helper functions. These utilities, `call_llm_robust`, `get_embedding`, and `query_pinecone`, are the vital communication lines that connect our engine's brain to its external tools. In [Chapter 4](#), they served their purpose well but relied on global variables for key components such as API clients and model names. As we move toward a production-ready system, this approach is too rigid and error-prone. These helper functions need to be hardened.

In this section, we will upgrade these core functions to be more reliable, testable, and transparent. The key architectural shift is the implementation of **dependency injection**. Instead of functions reaching for global variables, we will now explicitly pass all required components (such as clients and configurations) as arguments. This transforms them from simple connectors into resilient, self-contained modules that can handle failures gracefully, provide clear logging, and manage resources efficiently. Let's get started!

Enhancing modularity with dependency injection

The most significant upgrade to our helper functions is the move away from global variables. In our prototype, functions such as `call_llm_robust` implicitly knew which `client` and `GENERATION_MODEL` to use because they were defined in the global scope. This is convenient for a prototype but dangerous for a real application, as it creates hidden dependencies and makes the code difficult to test or reconfigure.

Our upgrade introduces **dependency injection**, a practice where a function's dependencies are "injected" as arguments. This makes each function a self-contained unit. It no longer matters *where* the client or model name comes from, as long as it's provided when the function is called. This makes our engine significantly more flexible. We could, for example, run two different engine tasks simultaneously using two different LLM providers just by passing in different client objects.

Improving transparency through production-level logging

In our prototype, we used `print()` statements to see what was happening. For a production system, this is insufficient. A professional system requires a formal log: a structured, timestamped, and reliable record of events.

Our final enhancement is to replace all `print()` statements with Python's built-in `logging` module. This is a fundamental shift that elevates our system's transparency. Proper logging allows us to differentiate

between routine `INFO` messages, potential warnings, and critical errors. This structured output is not only easier for humans to read but is also machine-readable, allowing logs to be fed into automated monitoring and alerting systems.

```
# === Configure Production-Level Logging ===
logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
```

Increasing efficiency with proactive context management

A critical aspect of building LLM applications is managing the "invisible cost" of tokens. Every piece of context we send to the model consumes tokens, which impacts cost and speed, and can lead to hard failures if the context window is exceeded. A professional system cannot be reckless with these resources.

To address this, we introduce the `count_tokens` utility. This function acts like a "fuel gauge," allowing the engine to measure the token cost of a prompt *before* sending it. It uses OpenAI's official `tiktoken` library for accuracy. By building this now, we lay the foundation for more advanced features later, such as automatically summarizing context to stay within a token budget.

```
# === Context Management Utility (New) ===
def count_tokens(text, model="gpt-4"):
    """Counts the number of tokens in a text string for a given model."""
    try:
        encoding = tiktoken.encoding_for_model(model)
    except KeyError:
        # Fallback for models that might not be in the tiktoken registry
        encoding = tiktoken.get_encoding("cl100k_base")
    return len(encoding.encode(text))
```

This simple utility is a crucial first step in making the engine not just powerful, but also cost-conscious and efficient.

Upgraded helper functions in action

Here is the final, hardened code for our core helper functions. Notice how each function signature now includes parameters such as `client`, `generation_model`, and `embedding_model`. Every `print` statement has been replaced by a `logging` call, and error handling is more specific.

```
# === LLM Interaction (Hardened with Dependency Injection) ===
@retry(wait=wait_random_exponential(min=1, max=60), stop=stop_after_attempt(6))
def call_llm_robust(
    system_prompt, user_prompt, client, generation_model,
    json_mode=False
):
    """
    A centralized function to handle all LLM interactions with retries.
    UPGRADE: Now requires the 'client' and 'generation_model' objects to be passed in.
    """
    logging.info("Attempting to call LLM...")
    try:
        response_format = {"type": "json_object"}
```

```

        if json_mode else {"type": "text"}
        # UPGRADE: Uses the passed-in client and model name for the API call.
        response = client.chat.completions.create(
            model=generation_model,
            response_format=response_format,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt}
            ],
        )
        logging.info("LLM call successful.")
        return response.choices[0].message.content.strip()
    except APIError as e:
        logging.error(f"OpenAI API Error in call_llm_robust: {e}")
        raise e
    except Exception as e:
        logging.error(f"An unexpected error occurred in call_llm_robust: {e}")
        raise e

# === Embeddings (Hardened with Dependency Injection) ===
@retry(wait=wait_random_exponential(min=1, max=60),
       stop=stop_after_attempt(6))
def get_embedding(text, client, embedding_model):
    """
    Generates embeddings for a single text query with retries.
    UPGRADE: Now requires the 'client' and 'embedding_model' objects.
    """
    text = text.replace("\n", " ")
    try:
        # UPGRADE: Uses the passed-in client and model name.
        response = client.embeddings.create(input=[text], model=embedding_model)
        return response.data[0].embedding
    except APIError as e:
        logging.error(f"OpenAI API Error in get_embedding: {e}")
        raise e
    except Exception as e:
        logging.error(f"An unexpected error occurred in get_embedding: {e}")
        raise e

```

By catching and logging a specific API error before re-raising the exception (`raise e`), we achieve the best of both worlds. The function reports a detailed, specific error in the logs, but it also signals the failure up to the main `context_engine` function. This allows the engine to make the strategic decision to halt the entire plan, a vital safety feature that prevents it from proceeding with incomplete information.

With these hardened helper functions, the engine's core communication layer is now modular and transparent, ready to support the more complex logic of our specialist agents.

Refactoring the agents for production

To give a complete picture of each agent, this section traces the full refactoring journey from the initial hardening within

`Context_Engine_MAS_MCP.ipynb` to the final modularized code that will reside in the `agents.py` library file. This approach lets us follow each agent's evolution and the real-world debugging process in one place, be-

fore we formally assemble the final library files in the *Modularizing the engine* section.

In this stage, we move away from global variables and instead pass all dependencies, such as API clients and model configurations, directly to the agents as arguments. This use of dependency injection makes the system easier to test, extend, and maintain. We also upgrade all internal communications from simple `print()` statements to structured logging, introducing the level of transparency expected in a production-grade environment.

Together, these changes transform the agents from simple task functions into transparent components of a business-ready application. Let's move on and see how we refactored `Context_Engine_MAS_MCP.ipynb` and adapted the updated code for deployment in `Context_Engine_Pre_Production.ipynb`, setting the stage for their integration in the *Modularizing the engine* section.

Context Librarian agent

Our first upgrade targets the Context Librarian. We will transform it from the simple prototype we built in [Chapter 4](#) into a hardened, production-ready component. The initial upgrade, performed inside the `Context_Engine_MAS_MCP.ipynb` notebook, focuses on introducing dependency injection and professional logging.

This agent's refactoring is a perfect case study for our production-hardening process. We will first examine the initial upgrades made within the notebook, then show you how the process of moving this code into the final `agents.py` file revealed a subtle but important flaw, and how we fixed it to create the final, production-ready version.

As we'll see, the first pass contained a critical flaw in the agent's output when the final version was executed. It's important to recognize that such issues are a natural part of developing AI agents and that solving them systematically is what makes a system resilient. The critical lines from the intermediate implementation show the agent's upgraded signature and its original, flawed `return` statement:

```
# === 4.1. Context Librarian Agent (Upgraded) ===
# *** Added 'namespace_context' argument ***
def agent_context_librarian(
    mcp_message, client, index, embedding_model, namespace_context
):
    # ... (logic for querying Pinecone) ...
    if results:
        # ...
        content = blueprint_json # FLAW: content is a raw string
        # ...
    return create_mcp_message("Librarian", content)
```

We implemented the following improvements:

- We improved the dependency injection by making sure the function signature now explicitly requires all its external needs: `client`, `index`, `embedding_model`, and `namespace_context`. This makes the agent a *self-contained, testable unit*.
- We also structured the logging process. All print statements have been replaced with professional logging calls.

- We developed a more robust error handling process. The entire logic is wrapped in a `try...except` block to handle failures gracefully.

During this process, the flaw in the earlier version became clear: it returned the blueprint as a raw JSON string.

For the system to be robust, every agent must return data in a predictable, structured way. The Writer agent, for instance, must know exactly which key to look for in the Librarian's output. The fix was to wrap the output in a dictionary with a consistent key, `"blueprint_json"`. This creates a stable data contract between agents.

The final, corrected version of the code, now residing in `agents.py`, reflects this crucial fix and the other professional upgrades.

Final hardened code

We now have the final code, broken down to show where each upgrade was implemented. First, we define the hardened function signature. All dependencies are now explicit arguments, not global variables, making the agent a self-contained unit:

```
def agent_context_librarian(
    mcp_message, client, index, embedding_model, namespace_context
):
    """Retrieves the appropriate Semantic Blueprint from the Context Library."""
```

Next, we replace the old `print()` statements with professional logging and wrap the entire function in a `try...except` block for robust error handling:

```
logging.info("[Librarian] Activated. Analyzing intent...")
try:
    requested_intent = mcp_message['content'].get('intent_query')
    if not requested_intent:
        raise ValueError("Librarian requires 'intent_query' in the input content.")
```

The call to the `query_pinecone` helper is now upgraded to use explicit keyword arguments and pass all the necessary client and model objects that were injected into the agent:

```
results = query_pinecone(
    query_text=requested_intent,
    namespace=namespace_context,
    top_k=1,
    index=index,
    client=client,
    embedding_model=embedding_model
)
```

Finally, we implement the most critical fix uncovered during the validation phase. The agent's output is now a dictionary with a consistent key, creating a reliable data contract for other agents to use:

```
if results:
    match = results[0]
    logging.info(
```

```

        f"[Librarian] Found blueprint '{match['id']}' (Score: {match['score']:.2f})"
    )
    blueprint_json = match['metadata']['blueprint_json']
    content = {"blueprint_json": blueprint_json}
else:
    logging.warning("[Librarian] No specific blueprint found. Returning default.")
    content = {"blueprint_json": json.dumps(
        {"instruction": "Generate the content neutrally."}
    )}
}
return create_mcp_message("Librarian", content)

```

This small change is the essence of refactoring: it's not just about moving code around but about making it more predictable and easier to maintain as the system grows.

Researcher agent

Similar to how we refined the Librarian agent, the Researcher follows the same pattern of incremental improvement, from notebook hardening to library integration. The process begins in

`Context_Engine_MAS_MCP.ipynb` with the introduction of dependency injection and structured logging, followed by modularization into the `agents.py` file.

As with the Librarian, the first pass of this agent exposed a subtle but critical flaw. During validation, we discovered that the agent returned raw strings instead of structured data, an easy mistake to make but one that can cascade through the entire workflow:

```

# === 4.2. Researcher Agent (Upgraded) ===
def agent_researcher(
    mcp_message, client, index, generation_model, embedding_model,
    namespace_knowledge
):
    # ... (logic for querying Pinecone and synthesizing) ...
    if not results:
        return create_mcp_message("Researcher", "No data found on the topic.") # FLAW: ret
    # ...
    findings = call_llm_robust(...)
    return create_mcp_message("Researcher", findings) # FLAW: returns a raw string

```

The upgrades in this stage were similar in structure to those applied to the Librarian:

- We strengthened dependency injection by explicitly requiring all external dependencies: `client`, `index`, `generation_model`, `embedding_model`, and `namespace_knowledge`
- We replaced `print()` statements with structured logging calls for production-level transparency
- We wrapped the logic in a `try...except` block for robust error handling

After hardening the agent within the notebook, the next step was to migrate it to the `agents.py` library file. During this process, the flaw in the earlier version became clear: it returned the blueprint as a raw JSON string.

For our system to be robust, however, agents must communicate using a predictable, structured format. The Writer agent needs to know exactly

what key to look for in the Researcher's output to find the facts. The solution is to wrap the output in a dictionary with a consistent key, "facts". This creates a stable *data contract* between the agents.

The final, corrected version of the code, as it appears in `agents.py`, reflects this crucial fix and other professional upgrades.

Final hardened code

Here is the final code, broken down to show where each upgrade was implemented. First, we define the hardened function signature. All dependencies are now explicit arguments, not global variables:

```
def agent_researcher(
    mcp_message, client, index, generation_model, embedding_model,
    namespace_knowledge
):
    """Retrieves and synthesizes factual information from the Knowledge Base."""
```

Next, we replace the old print statements with professional logging and wrap the entire function in a `try...except` block for robust error handling.

```
logging.info("[Researcher] Activated. Investigating topic...")
try:
    topic = mcp_message['content'].get('topic_query')
    if not topic:
        raise ValueError("Researcher requires 'topic_query' in the input content.")
```

The call to the `query_pinecone` helper is now upgraded. We use explicit keyword arguments and pass down all the necessary client and model objects that were injected into the agent.

```
results = query_pinecone(
    query_text=topic,
    namespace=namespace_knowledge,
    top_k=3,
    index=index,
    client=client,
    embedding_model=embedding_model
)
```

Finally, we implement the most critical fix. The agent's output is now a dictionary with a consistent "facts" key, creating a reliable data contract for other agents to use.

```
if not results:
    logging.warning("[Researcher] No relevant information found.")
    return create_mcp_message("Researcher", {"facts": "No data found on the topic."})

logging.info(f"[Researcher] Found {len(results)} relevant chunks. Synthesizing...")
source_texts = [match['metadata']['text'] for match in results]
system_prompt = """You are an expert research synthesis AI.
Synthesize the provided source texts into a concise, bullet-pointed summary answering the
user_prompt = f"Topic: {topic}\n\nSources:\n" + "\n\n---\n\n".join(source_texts)

findings = call_llm_robust(
    system_prompt,
```

```

    user_prompt,
    client=client,
    generation_model=generation_model
)
return create_mcp_message("Researcher", {"facts": findings})

```

By encapsulating outputs this way, we not only stabilized the data flow but also enforced consistency across the agent ecosystem. As we'll see next, the Writer agent, our final specialist, continues this same evolutionary path.

Writer agent

The Writer agent completes our trio of specialists, and its refactoring demonstrates the final integration of the system. Like the Librarian and Researcher, it was first hardened in `Context_Engine_MAS_MCP.ipynb` with structured logging, error handling, and dependency injection.

This upgrade, however, reveals the most important lesson about integrated systems: when you change one component, you must update the components that depend on it. Remember that there is no road from design to perfection. So, knowing how to solve issues is as important as developing functions! The initial hardened version of the Writer worked with the old Librarian and Researcher, but it contains a critical flaw that prevents it from working with their new, upgraded versions.

The critical lines from this intermediate implementation show the agent's flawed input handling:

```

# === 4.3. Writer Agent (Upgraded) ===
def agent_writer(mcp_message, client, generation_model):
    """Combines research with a blueprint to generate the final output."""
    logging.info("[Writer] Activated. Applying blueprint to source material...")
    try:
        # --- FIX: Unpack the structured inputs from previous steps ---
        blueprint_data = mcp_message['content'].get('blueprint')
        facts_data = mcp_message['content'].get('facts')
        previous_content_data = mcp_message['content'].get(
            'previous_content'
        )
    ...

```

Let's go through the initial upgrades:

- The function signature is hardened to explicitly require the client and `generation_model`, making it a self-contained unit
- All print statements are replaced with professional logging calls
- The entire logic is wrapped in a `try...except` block to handle failures gracefully.

Let's see how to fix the integration flaw.

The flaw lay in how the Writer handled its inputs. It expected raw strings, not structured dictionaries containing `"blueprint_json"` and `"facts"`. To fix this, we upgraded the Writer to unpack these structures intelligently. This fix is not just a bug fix; it's the final step in creating a truly interconnected and robust multi-agent system where data flows predictably

from one component to the next. The final, corrected version of the code, as it appears in `agents.py`, reflects this crucial integration fix.

Final hardened code

Here is the final code, broken down to show where each upgrade was implemented. First, we have the hardened function signature and initial logging setup:

```
def agent_writer(mcp_message, client, generation_model):
    """Combines research with a blueprint to generate the final output."""
    logging.info("[Writer] Activated. Applying blueprint to source material...")
    try:
```

Next, we implement the most critical fix. This new block of code intelligently unpacks the structured inputs, checking if they are dictionaries (from our new agents) or raw strings (for backward compatibility or other uses), making the agent robust:

```
        blueprint_data = mcp_message['content'].get('blueprint')
        facts_data = mcp_message['content'].get('facts')
        previous_content_data = mcp_message['content'].get(
            'previous_content'
        )

        # Extract the actual strings, handling both dict and raw string inputs
        blueprint_json_string = blueprint_data.get('blueprint_json')
        if isinstance(blueprint_data, dict) else blueprint_data
        facts = facts_data.get('facts')
        if isinstance(facts_data, dict) else facts_data
        previous_content = previous_content_data # Assuming this is already a string if pr

    if not blueprint_json_string:
        raise ValueError("Writer requires 'blueprint' in the input content.")
```

The rest of the agent's logic remains the same, but it now operates on the correctly unpacked facts and `blueprint_json_string` variables:

```
    if facts:
        source_material = facts
        source_label = "RESEARCH FINDINGS"
    elif previous_content:
        source_material = previous_content
        source_label = "PREVIOUS CONTENT (For Rewriting)"
    else:
        raise ValueError("Writer requires either 'facts' or 'previous_content'.")

    # ... (System and User prompt construction remains the same) ...
```

Finally, the agent makes its dependency-injected call to the LLM and returns the output, now confident that it has correctly interpreted the data from the preceding agents:

```
    final_output = call_llm_robust(
        system_prompt,
        user_prompt,
        client=client,
        generation_model=generation_model
```

```

    )
    return create_mcp_message("Writer", final_output)
except Exception as e:
    logging.error(f"[Writer] An error occurred: {e}")
    raise e

```

With the full team of specialist agents now upgraded to production standards, we can move on to refactoring the component that manages them.

Refactoring the Agent Registry

We now turn our attention to `AgentRegistry`, the component acting as our engine's foreman, holding the master clipboard. In [Chapter 4](#), it served as a simple roster, mapping agent names to their functions. Now, we will upgrade it to be a *smart* central toolkit.

Its upgrade is the final and most critical step in hardening our components, as it is responsible for correctly managing and injecting dependencies into all the specialized agents. Just like the other components, we will first upgrade it inside the `Context_Engine_MAS_MCP.ipynb` notebook. This initial upgrade focuses on enhancing the `get_handler` method to manage the newly required dependencies for our hardened agents (like client, index, and namespace configurations). However, as with the previous refactors, the first pass surfaced a fundamental architectural flaw. The notebook version referenced agent functions directly, a shortcut that only works within a single-file environment:

```

class AgentRegistry:
    def __init__(self):
        self.registry = {
            # ADD the "agents." prefix to each function name
            "Librarian": agent_context_librarian,
            "Researcher": agent_researcher,
            "Writer": agent_writer,
        }
        # ...

```

At this stage, the `get_handler` method was already significantly improved. Its signature now accepted all key dependencies (`client`, `index`, `generation_model`, and others) and used lambda functions to dynamically generate ready-to-use handlers for each agent, injecting only the dependencies required by that agent.

But once the registry was migrated into its own `registry.py` file, a familiar pattern emerged. Just as we saw when modularizing the agents, moving code out of the notebook exposed its isolation: each file is its own environment. The original code assumed that agent functions existed in the same global namespace, which caused the refactored module to crash with `NameError`.

The solution, like the fixes in the Librarian and Researcher agents, was to make the registry self-contained. It needed to explicitly import the `agents` module and reference each function using its correct namespace; for example, `agents.agent_context_librarian`. This small but crucial change ensures that every component knows exactly where its dependencies live, establishing a fully modular architecture.

The final, corrected version of the code in `registry.py` reflects this improvement.

Final hardened code

Here is the final code, broken down to show where each upgrade was implemented. First, the registry correctly imports its dependencies:

```
import logging
import agents
```

Next, inside the `__init__` method, it references the agent functions using the `agents.` prefix, resolving the modularity flaw:

```
class AgentRegistry:
    def __init__(self):
        self.registry = {
            # Add the "agents." prefix to each function name
            "Librarian": agents.agent_context_librarian,
            "Researcher": agents.agent_researcher,
            "Writer": agents.agent_writer,
        }
```

The `get_handler` method remains the same powerful dependency injector we built in the notebook, but it now operates on the correctly imported agent functions:

```
def get_handler(
    self, agent_name, client, index, generation_model,
    embedding_model, namespace_context, namespace_knowledge
):
    handler_func = self.registry.get(agent_name)
    if not handler_func:
        logging.error(f"Agent '{agent_name}' not found in registry.")
        raise ValueError(f"Agent '{agent_name}' not found in registry.")

    if agent_name == "Librarian":
        return lambda mcp_message: handler_func(
            mcp_message, client=client, index=index,
            embedding_model=embedding_model,
            namespace_context=namespace_context
        )
    elif agent_name == "Researcher":
        return lambda mcp_message: handler_func(
            mcp_message, client=client, index=index,
            generation_model=generation_model,
            embedding_model=embedding_model,
            namespace_knowledge=namespace_knowledge
        )
    elif agent_name == "Writer":
        return lambda mcp_message: handler_func(
            mcp_message, client=client,
            generation_model=generation_model
        )
    else:
        return handler_func
```

That was quite a ride from design to development and fixing flaws. Let's upgrade the context engine next.

Upgrading the central orchestrator

With the agents and registry now hardened and modularized, we turn our attention to the heart of the architecture: the Context Engine itself. Acting as the system’s central “brain,” it brings all individual components together into a cohesive, thinking machine.

In [Chapter 4](#), we built the core elements that powered this orchestration loop. In this section, we’ll upgrade each of them to meet our new production-ready standards. The Context Engine serves as the central nervous system of the entire application, containing three critical components: the `ExecutionTrace` class (the Tracer), the `planner` function, and the `context_engine` orchestrator.

Its migration from `Context_Engine_MAS_MCP.ipynb` to a dedicated `engine.py` file marks the final and most significant step in our transformation toward a modular, pre-production system. While the individual classes, such as `ExecutionTrace`, were already hardened in the notebook, the monolithic structure that tied them all together remained a key flaw. For the engine to be reusable and maintainable, its logic must be fully decoupled from the notebook environment.

As we’ve seen with the other components, moving from a single-notebook prototype to a modular code base exposes implicit dependencies. The notebook version of the Context Engine assumed that all supporting modules (agents, helpers, and the registry) existed in the same scope. Once we move the engine into its own file, that assumption breaks:

```
class ExecutionTrace:
    # ... (Tracer logic is already robust) ...

def planner(goal, capabilities):
    # ...

def context_engine(goal, ...):
    # FLAW: This function directly calls other functions like planner()
    # and uses the AGENT_TOOLKIT, assuming they are globally available.
```

Following the same process we applied to the agents and the registry, the engine is now moved into its own file, `engine.py`. This modularization not only fixes the architectural flaw but also establishes the Context Engine as a self-contained component that explicitly imports everything it depends on.

The `engine.py` file now imports the helpers and the registry (which itself imports the agents). The result is a standalone engine that can be imported into any notebook or integrated directly into larger applications, achieving our goal of a truly pre-production, reusable system.

The final, corrected version of the code shown next represents a complete, end-to-end orchestration layer that binds all components together.

Final hardened code

Here is the final code, broken down to show its key components. First, we have the modular imports, which fix the core architectural flaw of the monolithic notebook.

```
...
import logging
import time
import json
import copy
from helpers import call_llm_robust, create_mcp_message
from registry import AGENT_TOOLKIT
```

Next is the `ExecutionTrace` class. Its code was already robust, so it is moved directly into the module without changes.

```
class ExecutionTrace:
    """Logs the entire execution flow for debugging and analysis."""
    # ... (Full class code as provided) ...
```

The `planner` function is also moved into the module. It acts as the strategic core, using the LLM to generate a step-by-step plan.

```
def planner(goal, capabilities, client, generation_model):
    """Analyzes the goal and generates a structured Execution Plan using the LLM."""
    # ... (Full planner logic as provided) ...
```

Finally, the `context_engine` function serves as the main orchestrator or Executor. It initializes the trace, calls the planner, and then loops through each step of the plan, using `AGENT_TOOLKIT` to get the correct agent handler and execute the task. This is the heart of the engine that brings the entire plan to life.

```
def context_engine(
    goal, client, pc, index_name, generation_model, embedding_model,
    namespace_context, namespace_knowledge
):
    """The main entry point for the Context Engine."""
    # ... (Full execution loop logic as provided) ...
```

With the engine's brain now fully upgraded, the entire framework is hardened and ready. The final step in this chapter is to test our newly refactored system with a live execution run.

Running the hardened engine

We have successfully refactored and hardened every component of our Context Engine. We've upgraded our helpers for reliability, modularized our agents with dependency injection, created a smart Agent Registry, and consolidated our core logic into the `context_engine` orchestrator. Now, the moment of truth has arrived. It's time to initialize this entire system and observe it in action.

In this final section, we will run the fully assembled engine. We are not just testing if it works; we are demonstrating that our architectural upgrades have resulted in a system that is transparent and capable of dynamic, multi-step reasoning.

Visualizing the trace

Our `engine.py` module is now complete. However, there's one final utility to add to our notebook environment. Logs provide a detailed record for auditing, but a quick, visual summary of the engine's behavior is invaluable for immediate analysis.

To achieve this, we will keep our `engine.py` library clean and define a dedicated presentation function inside our notebook called `execute_and_display`. This function acts as our "engine room," handling the entire process: it calls the engine, receives the final result and the detailed trace object, and then formats both for a clean, human-readable display. This approach strictly separates the core engine's logic from the user-facing output.

It will be integrated into the *The final pre-production notebook* section. Let's now run a standard execution.

Standard workflow execution

Our first test uses a simple, familiar goal that is similar to the example from [Chapter 4](#). We'll ask the engine to write a suspenseful scene, which should naturally trigger the classic Librarian → Researcher → Writer workflow.

The goal of this test is to confirm that the refactored Context Engine can correctly assemble and execute a basic multi-agent plan from start to finish. We'll define the goal, run the engine, and if the process completes successfully, use our new `display_trace()` method to visualize exactly how the system achieved the result:

```
#Example 1
# Define the high-level goal
goal = "Write a short, suspenseful scene for a children's story about the Apollo 11 moon l
# Call the execution function from the cell above
execute_and_display(goal, config, client, pc)
```

The output first displays the story:

```
--- FINAL OUTPUT ---
July 20, 1969. The cabin hums. The Moon fills the window. Gray. Still. Strange.
I'm in Eagle with Buzz. First try. No second chances.
The small computer chirps. 1202. Then 1201. My chest tightens. We ask. Voices crackle in m
Shadows stretch across the ground. Big. Long...
```

Then we get the trace as requested:

```
--- TECHNICAL TRACE (for the tech reader) ---
Trace Status: Success
Total Duration: 148.33 seconds
Execution Steps:
[ { 'agent': 'Librarian',
  'output': { 'blueprint_json': '{"scene_goal": "Increase tension and create '...
```

Complex workflow execution

Our second test pushes the Planner beyond a simple, linear workflow. This time, we'll give it a more sophisticated goal: first, write a factual

summary of the Apollo 11 moon landing; then, rewrite that same content in the minimalist, impactful style of Ernest Hemingway.

This task cannot be solved with the usual **Librarian** → **Researcher** → **Writer** sequence. A truly intelligent Planner must recognize it as a two-stage process:

- **Stage 1 – Generation:** Gather facts and produce the initial description.
- **Stage 2 – Rewriting:** Retrieve a Hemingway style blueprint and apply it to the output of *Stage 1*.

This scenario compels the Planner to chain two Writer agents together, using the output of the first as the input to the second. It's a powerful demonstration of context chaining, proof that the engine isn't following a fixed script, but dynamically reasoning through the steps required to achieve the goal. You can change the goal in the notebook:

```
goal_2 = "First, write a factual summary of the Apollo 11 landing. Then, rewrite that summ
...
```

The output provides a sharp Hemingway journalist-style response:

```
Here's the quick-and-casual lowdown on Apollo 11
- First crewed Moon landing, round trip in about 8 days. Launched on a Saturn V from Kenne
- Dates: Launch July 16, 1969; landing July 20 (20:17:40 UTC); first step July 21 (02:56:1
- Crew: Neil Armstrong (Commander), Buzz Aldrin (Lunar Modul
```

Then the trace also:

```
--- TECHNICAL TRACE (for the tech reader) ---
Trace Status: Success
Total Duration: 151.31 seconds
Execution Steps:
[ { 'agent': 'Researcher',
  'output': { 'facts': 'Note: The provided "Sources" are unrelated to Apollo '...
```

Now that we've run both workflows, it's time to look beneath the surface and see how the engine actually reasoned through each step. The best way to do that is by examining the **execution log**.

Deconstructing the engine's thought process

The execution log is a real-time transcript of the Context Engine's operations. It reveals how the system converts a high-level goal into a structured sequence of concrete actions, demonstrating, in detail, how planning, execution, and context chaining unfold inside the engine.

Initiation and planning

```
=== [Context Engine] Starting New Task ===
Goal: Write a short, suspenseful scene for a children's story about the Apollo 11 moon lan

[Engine: Planner] Analyzing goal and generating execution plan...
[Engine: Planner] Plan generated successfully.
```

This initial block confirms the engine has received the `goal`. The Planner immediately activates, analyzes the request, consults the Agent Registry for available tools, and formulates a strategic, three-step JSON plan to achieve the goal. This is the *thinking* phase of the engine.

Execution step 1 – The Librarian

```
[Engine: Executor] Starting Step 1: Librarian
[Librarian] Activated. Analyzing intent...
[Librarian] Found blueprint 'blueprint_suspense_narrative' (Score: 0.66)
[Engine: Executor] Step 1 completed.
```

The Executor now begins to act on the plan. It activates the Librarian agent with the input `intent_query: "suspenseful children's story"`. The Librarian, in turn, converts this query into a vector and searches the `ContextLibrary` namespace in Pinecone. It finds a matching blueprint named `'blueprint_suspense_narrative'`, which contains instructions on how to write with a suspenseful tone (e.g., use short sentences, focus on sensory details, build tension).

Execution step 2 – The Researcher

```
[Engine: Executor] Starting Step 2: Researcher
[Researcher] Activated. Investigating topic...
[Researcher] Found 2 relevant chunks. Synthesizing...
[Engine: Executor] Step 2 completed.
```

Next, the Executor activates the Researcher agent with the input `topic_query: "Apollo 11 moon landing danger"`. The Researcher searches the `KnowledgeStore` namespace and retrieves two chunks of factual text related to the Apollo 11 mission. It then uses an LLM to synthesize these facts into a concise summary, which becomes its output.

Execution step 3 – The Writer

```
[Engine: Executor] Starting Step 3: Writer
[Engine: Executor] Resolved dependency STEP_1_OUTPUT.
[Engine: Executor] Resolved dependency STEP_2_OUTPUT.

[Writer] Activated. Applying blueprint to source material...
[Engine: Executor] Step 3 completed.
```

This is the final and most critical step. The Executor first performs context chaining. It resolves the `$$STEP_1_OUTPUT$$` placeholder with the suspense blueprint from the Librarian and the `$$STEP_2_OUTPUT$$` placeholder with the factual summary from the Researcher. The Writer agent is then activated, receiving both the *style instructions* and the *factual content* as its inputs. It combines them to generate the final text.

Finalization

With the final step complete, the engine's task concludes successfully.

```
=== [Context Engine] Task Complete ===
```

The final output is a direct product of the three-agent workflow. It's a synthesis of the requested style and the verified facts, working in perfect harmony:

```
***** FINAL OUTPUT 1 *****
July 20, 1969. Apollo 11. NASA's Apollo program. The Space Race hums in the background lik
Three of us. Neil Armstrong. Buzz Aldrin. Michael Collins.
The Moon fills the window. Black sky. Hard light. Long shadows.
...
The hum rises, falls. The Eagle steadies. The scene tightens to a pin.
Then—stillness.
...
A boot hovers. I watch the shadow first. It kisses the surface before I do.
Then contact. A first step. On the Moon.
...
The moment stretches thin, like a note, and does not break.
```

Let's analyze the output:

- **Influence of the Librarian (style):** The output perfectly matches `'blueprint_-suspense_narrative'`. The sentences are short, declarative, and impactful ("The cabin is small." "Fuel is low."). It focuses on sensory details ("A soft hiss in my ears," "A tremor underfoot") and uses evocative, tension-building language ("The scene tightens to a pin," "The moment stretches thin...and does not break"). This is the blueprint's stylistic guidance in action.
- **Influence of the Researcher (facts):** The story is grounded in the factual information retrieved by the Researcher. It correctly mentions the date (July 20, 1969), the mission name (Apollo 11), the crew (Armstrong, Aldrin, Collins), and the name of the lunar module (Eagle).

This output clearly demonstrates that the hardened Context Engine is working as intended. It successfully planned a dynamic workflow, orchestrated multiple agents, and produced a cohesive result through context chaining. We've officially evolved from a clever prototype into a robust, resilient, and transparent system. The next step is to make it truly seamless to use.

Modularizing the engine

With every component of the Context Engine now hardened, we can take the final, and most significant, step in our journey from prototype to pre-production: refactoring for modularity. In this phase, we'll transform the single, monolithic notebook into a clean, scalable application by moving the core logic into separate Python library files.

This process is more than a simple copy-and-paste exercise. It's a practical debugging journey that reveals how Python modules communicate, manage dependencies, and exchange data. We'll begin with a clean notebook, `Context_Engine_Pre_Production.ipynb`, which serves as our control deck, and organize our hardened code into a dedicated `commons/` library.

Our goal is to separate the code from sections 3, 4, 5, and 6 in the `Context_Engine_MAS_MCP.ipynb` notebook into their own `.py` files, `helpers.py`, `agents.py`, `registry.py`, and `engine.py`, which will reside inside a new `commons/` directory. This restructuring marks the final

step in transforming our Context Engine into a modular, scalable application.

Local imports

Our first challenge is environmental. An `import` statement for a structured project (`from commons import helpers`) will fail in a flat Colab directory where we download all files to the same level. We must adapt our imports to match this flat structure, treating each file as a top-level module.

The modified code for Colab will be as follows:

```
# This works in a flat directory by importing each file directly.
import helpers
import agents
from registry import AGENT_TOOLKIT
from engine import context_engine
```

With this, our notebook can see the new library files. However, the libraries can't yet see each other.

Module independence

Running the new import cell immediately reveals a critical lesson: every Python file is an island. A module knows nothing of the outside world, including other modules, unless it is explicitly told.

Issue 1: Missing agents

The first error occurs when importing `registry.py` :

```
NameError: name 'agent_context_librarian' is not defined
```

The `registry.py` file's purpose is to know about all the agents. However, the agent functions themselves are now defined in a different file, `agents.py`. The registry, being an isolated module, has no idea that the `agents.py` file exists.

The solution is that we must explicitly connect the modules. We edit `registry.py` to import the `agents` module and then prefix each function call with `agents.`, making it clear where the function lives.

The modified code in `registry.py` will be as follows:

```
# Tell this file about the agents module
import agents
import logging

class AgentRegistry:
    def __init__(self):
        self.registry = {
            # Specify where to find each function
            "Librarian": agents.agent_context_librarian,
            "Researcher": agents.agent_researcher,
            "Writer": agents.agent_writer,
```

```
}
# ... rest of the class
```

This process of identifying and importing dependencies continues for all our files. The `engine.py` file needs to import our `helpers` and `registry` modules, while `agents.py` needs to import functions from `helpers.py`. This methodical debugging is a perfect illustration of building a truly modular system where every component explicitly declares what it needs.

Issue 2: Mismatch of data structures

After resolving all import errors, the engine runs but crashes at the final step. The error is more subtle this time: our agents are passing structured dictionaries (e.g., `{'blueprint_json': '...'}`) between steps, but the `Writer` agent expects a raw string for its source material. The final output from the `Writer` is not a story, but an error.

The fix is to make the `Writer` agent more intelligent. We modify it to check if its inputs (`blueprint` and `facts`) are dictionaries. If they are, it extracts the string value from the correct key before proceeding. This ensures that, no matter how the data is structured internally, the agent gets the raw text it needs to work.

The modified code in `agent_writer` (inside `agents.py`) will be as follows:

```
# --- FIX: Unpack the structured inputs from previous steps ---
blueprint_data = mcp_message['content'].get('blueprint')
facts_data = mcp_message['content'].get('facts')
previous_content_data = mcp_message['content'].get('previous_content')

# Extract the actual strings, handling both dict and raw string inputs
blueprint_json_string = blueprint_data.get('blueprint_json')
if isinstance(blueprint_data, dict)
    else blueprint_data
facts = facts_data.get('facts')
if isinstance(facts_data, dict)
    else facts_data
previous_content = previous_content_data # Assuming this is already a string if provided
```

With this final fix, our refactoring journey is complete. We have successfully transformed a linear notebook into a professional, modular application. We can now build our final notebook.

The final pre-production notebook

Our final notebook, `Context_Engine_Pre_Production.ipynb`, is now incredibly clean and serves as the high-level "control deck" for our powerful engine. It has just two main parts.

Centralized execution function (the engine room)

The first section defines a single function, `execute_and_display()`, that encapsulates all the logic for running the engine and presenting its results. It takes the user's goal, a configuration dictionary, and the initial-

ized clients, then prints both a polished final output for the user and a detailed technical trace for the developer.

```
# === ENGINE ROOM: The Main Execution Function ===
# This function contains all the logic to run the engine.
# We define it here so our final cell can be very simple.

import logging
import pprint
from IPython.display import display, Markdown

def execute_and_display(goal, config, client, pc):
    """
    Runs the context engine with a given goal and configuration,
    then displays the final output and the technical trace.
    """
    logging.info(f"***** Starting Engine for Goal: '{goal}' *****\n")

    # 1. Run the Context Engine using the provided configuration
    result, trace = context_engine(
        goal,
        client=client,
        pc=pc,
        **config # Unpack the config dictionary into keyword arguments
    )

    # 2. Display the Final Result for the main reader
    print("--- FINAL OUTPUT ---")
    if result:
        display(Markdown(result))
    else:
        print(f"The engine failed to produce a result. Status: {trace.status}")

    # 3. Display the Technical Trace for the developer/technical reader
    print("\n\n--- TECHNICAL TRACE (for the tech reader) ---")
    if trace:
        print(f"Trace Status: {trace.status}")
        print(f"Total Duration: {trace.duration:.2f} seconds")
        print("Execution Steps:")
        # Use pprint for a clean, readable dictionary output
        pp = pprint.PrettyPrinter(indent=2)
        pp.pprint(trace.steps)
```

We now have a centralized execution process. Let's explore user interactions.

User interactions (the control deck)

The final cell is the only one the user needs to interact with. Here, we define the high-level goal and a `config` dictionary that holds all the technical parameters for the run (such as model names and Pinecone namespaces). This clean separation of concerns, placing logic in the engine room and user inputs in the control deck, is the hallmark of a well-designed application.

The configuration is in a cell above the user goal:

```
# 1. Define all configuration variables for this run in a dictionary
config = {
    "index_name": 'genai-mas-mcp-ch3',
    "generation_model": "gpt-5",
```

```
"embedding_model": "text-embedding-3-small",
"namespace_context": 'ContextLibrary',
"namespace_knowledge": 'KnowledgeStore'
}
```

The control deck is now optimized:

```
#Example 1
# Define the high-level goal
goal = "[YOUR GOAL]"
# Call the execution function from the cell above
execute_and_display(goal, config, client, pc)
```

The system generates the same output structure as in the *Running the hardened engine* section. We have built a solid pre-production context engine. Let's summarize our journey and move on.

Summary

In this chapter, we elevated the Context Engine from a functional prototype to a pre-production system. By refactoring and rebuilding its architecture, we enforced professional engineering principles that made the engine robust, transparent, and reliable, a practical blueprint for developing resilient multi-agent systems.

The most impactful change was the introduction of dependency injection, which removed reliance on global variables and made every component self-contained and explicitly aware of its dependencies. This decoupled design dramatically improved the engine's flexibility, testability, and maintainability.

We also hardened each subsystem, from helper functions to agents, by adding structured logging and precise error handling. These upgrades created a fully auditable, fault-tolerant workflow. Finally, we demonstrated how to refactor a monolithic notebook into a modular application, showing how separate library files and clear inter-module communication create a scalable project structure.

The result is a professionally architected Context Engine with a clean separation between the user-facing control deck and the modular core. It now operates as a collection of specialized, independent components orchestrated through a predictable, transparent workflow.

In the next chapter, we will begin applying our context engine to real-world implementation issues and use cases.

Questions

1. Is the main focus of this chapter to build the initial prototype of the Context Engine from scratch? (Yes or no)
2. During the Planning phase, does the `planner()` function immediately call the specialist agents to get work done? (Yes or no)
3. Is the `resolve_dependencies()` function's main role to handle context chaining by replacing placeholders like `$$STEP_1_OUTPUT$$` with actual data? (Yes or no)
4. Is the Agent Registry described as the *brain* of the system that manages the entire process? (Yes or no)

5. Was the `utils.py` file created during refactoring to handle the high-level orchestration and planning logic of the engine? (Yes or no)
6. Was the main logging method in the original **Chapter 4** prototype the use of Python's built-in logging module? (Yes or no)
7. When refactoring the Librarian and Researcher agents, was a key flaw identified where they returned raw strings instead of structured dictionaries? (Yes or no)
8. When moving the code from the notebook to separate `.py` files, did the `registry.py` file crash because it could no longer find the agent functions? (Yes or no)
9. Did the *Complex workflow execution* test (rewriting in Hemingway's style) confirm that the engine always follows a fixed Librarian -> Researcher -> Writer sequence? (Yes or no)

References

- Qian, C., Wu, Y., Zhang, J., Wu, Y., Ahmed, J., Liu, B., & Lo, D. (2024). LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. arXiv preprint arXiv:2404.04834 *arXiv preprint arXiv:2404.04834*. <https://arxiv.org/abs/2404.04834>
- Zhang, Y., Li, R., Liang, P., Sun, W., & Liu, Y. (2025). Knowledge-Based Multi-Agent Framework for Automated Software Architecture Design. arXiv preprint arXiv:2503.20536 *arXiv preprint arXiv:2503.20536*. <https://arxiv.org/abs/2503.20536>
- Zhang, W., Wang, Z., Li, Y., Chen, Z., Zhang, Z., Liu, S., ... & Liu, B. (2025). AgentOrchestra: A Hierarchical Multi-Agent Framework for General-Purpose Task Solving. arXiv preprint arXiv:2506.12508 *arXiv preprint arXiv:2506.12508*. <https://arxiv.org/abs/2506.12508>
- Cito, J., Kassab, M., & Parnin, C. (2021). Cataloging Dependency Injection Anti-Patterns in Software Systems. arXiv preprint arXiv:2109.04256 *arXiv preprint arXiv:2109.04256*. <https://arxiv.org/abs/2109.04256>

Further reading

Wooldridge, M. (2009). An Introduction to Multiagent Systems (2nd ed.). John Wiley & Sons.

Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require one.

Table of contents collapsed